RL Fine-Tuning of Language Models

CS 224R Default Project Specification

You will explore how to use RL for post-training large language models (LLMs) through two main parts: (1) a core implementation section where you will get the chance to implement parts of the reinforcement learning stack for LLMs, and (2) a research-oriented extension where you explore a research direction of your choosing and evaluate whether it improves downstream performance.

The default implementation is organized around the Countdown arithmetic reasoning task with the Qwen model. You will implement three core stages: (1) supervised fine-tuning (SFT) for warm-starting, (2) pairwise preference optimization using a DPO/IPO-style objective, and (3) online policy-gradient optimization with RLOO using a rule-based verifier reward. Next, you will construct an evaluation setup to rigorously compare these stages against relevant baselines. Finally, you will propose an extension of your choosing to explore how existing algorithmic ideas can be improved and stress-tested.

Note on Clarifications

Please use the default project Ed thread for questions and clarifications: https://edstem.org/us/courses/97036/discussion/7926481. Course staff will use this thread to answer questions and share clarifications with students.

Note on default project vs. custom project:

It is not intended for the default final project to require less effort or work than the custom final project. The default project simply reduces the overall difficulty of devising your own project idea and evaluation methods, allowing students to commit an equivalent amount of effort to the provided problem.

Note on Default Project Tutorial

We will have a virtual tutorial about the default project to walk through the details of the project and the starter code. Look out for announcements regarding this on Ed.

Note on Implementation Integrity

Please do not use/refer to code from any existing codebase, framework, or AI agent (e.g., Cursor, Codex, Claude Code, Antigravity, etc.) for ANY part of the default project except for the final extension. You may use required infrastructure libraries such as Hugging Face for model, tokenizer, and dataset loading, but you may not use high-level trainers such as `SFTTrainer` for the implementation component. The Stanford Honor Code applies!

Note on OAE Extensions

Assignment extensions granted by the Office of OAE do not apply to group assignments. However, if you are working individually they can apply.

Note on Assistance

TAs may not look at students' code for either the default or custom final projects.

Contents

1. Initial Implementation

1.1. Background

Reinforcement Learning is a popular paradigm for LLM fine-tuning today. Two dominant approaches are prevalent today: (1) training on preference data containing relative feedback, helpful for tasks where a ground truth reward is hard to capture but can rank two responses well (e.g grading an essay might be quite tough but it might be easier to differentiate a lower quality essay from a higher quality essay), and (2) training with verifier-based rewards, used for tasks such as math and code reasoning. Below, we will give some background on each paradigm.

Let's start with learning from preference data. Typically, before training on preference data, a pre-trained model is fine-tuned on high-quality data from the task of interest via supervised fine-tuning (SFT), to obtain a "reference" model π_{ref}. Then, to fine-tune π_{ref} with human preferences, usually a preference dataset $\mathcal{D}_{\text{pref}} = \{\mathbf{x}^{(i)}, \mathbf{y}_w^{(i)}, \mathbf{y}_l^{(i)}\}$ is collected, where $\mathbf{x}^{(i)}$ denotes a prompt and $\mathbf{y}_w^{(i)}, \mathbf{y}_l^{(i)}$ denote preferred and dispreferred responses, often obtained by sampling from π_{ref}. Given a preference dataset, most fine-tuning pipelines assume the existence of an underlying reward function $r^*(\mathbf{x}, \cdot)$. One popular framework for this is the Bradley-Terry (BT) model [4], assuming that human preferences can be written as:

$$p^*(\mathbf{y}_1 \succ \mathbf{y}_2|\mathbf{x}) = \frac{e^{r^*(\mathbf{x},\mathbf{y}_1)}}{e^{r^*(\mathbf{x},\mathbf{y}_1)}+e^{r^*(\mathbf{x},\mathbf{y}_2)}} \tag{1}$$

Given this reward function r^*, preference fine-tuning aims to find the optimum of the reward r^*. While the ultimate goal of preference fine-tuning is to find the *unconstrained* optimum of the reward function, in practice, we often replace the reward function with a reward model. Since the reward model is erroneous, we apply a KL-divergence constraint on the policy to prevent exploitation in the reward model. To align our results with typical preference fine-tuning procedures, we will consider such a KL-constrained reward optimization as our fine-tuning goal:

$$\max_{\pi_\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}_{\text{pref}},\mathbf{y}\sim\pi_\theta(\cdot|x)}[r^*(\mathbf{x},\mathbf{y})] - \beta\mathbb{D}_{\text{KL}}[\pi_\theta(\cdot|\mathbf{x})||\pi_{\text{ref}}(\cdot|\mathbf{x})] \tag{2}$$

The regularizer, weighted by β, controls the deviation of π from π_{ref} under the reverse KL divergence.

Reward model training. In order to fine-tune an LLM policy $\pi_\theta(\mathbf{y}|\mathbf{x})$, Eq. (1) provides a convenient way to learn a reward model either explicitly (i.e., by fitting a parametric reward model $r_\phi(\mathbf{x}, \mathbf{y})$) or implicitly (i.e., via direct preference optimization (DPO) [25] or IPO [14], that re-purposes the log-likelihood $\log \pi_\theta(\mathbf{y}|\mathbf{x})$ of the policy to represent the reward $r_\theta(\mathbf{x}, \mathbf{y})$). Explicit reward models are trained using the following classification objective:

$$\max_\phi \ \mathbb{E}_{(\mathbf{x},\mathbf{y}_w,\mathbf{y}_l)\sim\mathcal{D}_{\text{pref}}} \left[\log \sigma \left(r_\phi(\mathbf{x},\mathbf{y}_w) - r_\phi(\mathbf{x},\mathbf{y}_l)\right)\right], \tag{3}$$

where σ is the logistic function. Contrastive learning objectives [25, 14] on the other hand repurposes $\log \pi_\theta(\mathbf{y}|\mathbf{x})$ as the implicit reward $r_\theta(\mathbf{x}, \mathbf{y})$:

$$r_\theta(\mathbf{x},\mathbf{y}) = \beta \left[\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \log \pi_{\text{ref}}(\mathbf{y}|\mathbf{x})\right]. \tag{4}$$

Policy learning. On-policy RL approaches such as PPO [26] and REINFORCE [34] explicitly sample new responses from the current snapshot of the learned policy, $\mathbf{y}_i \sim \pi_\theta(\cdot|\mathbf{x}_i)$, score them under the reward model, and perform a policy gradient update on parameters θ, for example:

$$\theta' \leftarrow \theta - \eta\mathbb{E}_{\mathbf{x}\sim\mathcal{D}_{\text{pref}},\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})} \left[\nabla_\theta \log \pi_\theta(\mathbf{y}|\mathbf{x}) \cdot \bar{r}_\phi(\mathbf{x},\mathbf{y})\right] \tag{5}$$

is the gradient update employed by REINFORCE, where $\bar{r}_\phi(\mathbf{x}, \mathbf{y})$ corresponds to a normalized estimate of the reward model's predictions over a batch of samples drawn from the policy. Using a normalized reward estimate instead of directly the raw reward value helps reduce the variance of the policy gradient estimate as seen in RLOO [1]. High variance gradients slow down convergence and even sometimes lead to sub-optimal solutions in deep RL [20].

For more details:

Here is an additional set of references, looking at RLHF and RL optimization of LLMs:

1. CS 336 Lecture Notes: https://stanford-cs336.github.io/spring2024/
2. CSE 291 (AI Agents): https://pearls-lab.github.io/ai-agents-course/index.html

1.2. Description of Tasks and Dataloading

This section describes the default task used throughout the project and explains how the data pipeline differs across the training algorithms. Although alignment methods are often studied on broad instruction-following tasks, in this project we focus on a single reasoning task—*Countdown*—so that SFT, preference optimization, and online RL can be compared in a controlled setting using the same underlying problem domain.

1.2.1. Default Task: Math Reasoning with Countdown

Math Reasoning (Countdown) Countdown, as studied in Countdown [12], evaluates a model's ability to solve multi-step mathematical reasoning problems expressed in natural language. Each example provides a set of numbers and a target value; the model must generate a sequence of arithmetic operations that transforms the given numbers into the target. This setting tests whether the model can plan, decompose a problem into intermediate steps, and execute the required arithmetic reliably.

1.2.2. How Data Depends on the Training Algorithm

Different training algorithms require different dataset structures. In this project, we use two main dataset types:

1. **Preference datasets**, used for preference-based methods such as DPO or IPO.
2. **Verifier-based datasets**, used for online RL methods such as RLOO.

In general, preference datasets are more commonly used for instruction-following and helpfulness-style alignment than for mathematical reasoning. However, in this project we also use a preference dataset for Countdown so that all methods can be studied on a single task and compared more directly.

For each algorithm, the starter code provides a data pipeline that loads a Hugging Face dataset (https://huggingface.co/docs/datasets/en/index) and produces batches suitable for training and evaluation.

1.2.3. Preference Datasets

What they contain Preference datasets are typically structured as triples consisting of a prompt, a preferred response, and a dispreferred response. In this project, the Countdown preference dataset

follows this general pattern, even though the underlying task is mathematical reasoning rather than open-ended instruction following.

A standard way to represent these examples is with a chat template (https://huggingface.co/docs/transformers/main/en/chat_templating#advanced-adding-and-editing-chat-templates), which formats the conversation context consistently during training and evaluation.

How they are loaded To train language models, text must be tokenized using the tokenizer associated with the pretrained model, converting prompts and responses into token sequences. In this project, we use Hugging Face tokenizers (https://huggingface.co/learn/nlp-course/en/chapter2/4) and apply padding and truncation to form fixed-size batches (see https://huggingface.co/docs/transformers/en/pad_truncation).

For instruction-following and reasoning objectives, it is common to apply the loss only to completion tokens and not to prompt tokens. Accordingly, dataset construction may include an attention mask or loss mask that excludes query tokens from loss computation.

Efficient data loading is important to avoid training bottlenecks. A natural choice is a PyTorch `DataLoader` (https://pytorch.org/tutorials/beginner/basics/data_tutorial.html), which samples mini-batches for loss computation and backpropagation. Hugging Face datasets are directly compatible with Torch dataloaders (https://huggingface.co/docs/datasets/v1.13.0/use_dataset.html).

1.2.4. Verifier-Based Datasets

What they contain Verifier-based datasets are typically structured as a corpus of problems together with target answers or metadata needed to verify correctness. For Countdown, each example contains a set of numbers and a target value; the model must generate operations (e.g., $+$, $-$, $\times$, $/$) that evaluate to the target.

Unlike preference datasets, verifier-based datasets do not require preferred/dispreferred response pairs. Instead, they support *online* generation: the model samples candidate solutions during training, and each sampled solution is scored by a verifier or reward function.

When they are used These datasets are used for online RL methods such as RLOO. During training, the model samples rollouts on-policy from the prompt set, and the verifier determines whether each rollout is correct.

Rule-based reward function For verifier-based training, the repository provides a local reward implementation in `evaluation/countdown.py`. This reward function checks whether the generated output follows the required `<answer>...</answer>` format and whether the arithmetic is correct with valid number usage. This reward is therefore specific to verifier-based methods such as RLOO, and is not used for preference-based methods like DPO or IPO.

1.2.5. Datasets Used in the Default Countdown Pipeline

The default Countdown pipeline uses the following public datasets:

- **Warm-start SFT dataset**: https://huggingface.co/datasets/Asap7772/cog_behav_all_strategies
- **Official SFT checkpoint** (optional warm-start for IPO/RLOO): https://huggingface.co/asingh15/qwen-sft-countdown-defaultproj

- **Pairwise preference dataset** (for preference-based training such as IPO): https://huggingface.co/datasets/asingh15/countdown_tasks_3to4-dpo
- **Prompt/evaluation dataset** (for verifier-based online RL such as RLOO): https://huggingface.co/datasets/asingh15/countdown_tasks_3to4

Dataset Setup

For the default Countdown pipeline, the role of each dataset is:

- **Warm-start SFT dataset**: used to initialize the model before alignment.
- **Preference dataset**: used by preference-based optimization methods that compare chosen and rejected responses.
- **Prompt/evaluation dataset**: used by verifier-based online RL methods, where the model generates rollouts that are scored by the rule-based reward.

1.3. Method Implementation (Difficulty: Medium, Time: High)

In this section, you will implement a set of algorithms for preference-based and verifier-based RL fine-tuning. For all experiments, you **must use the Qwen 2.5 0.5B Base model (https://huggingface.co/Qwen/Qwen2.5-0.5B)**. You **may not use any other model** (including an instruct variant) as the initialization for fine-tuning; this constraint is required for fair evaluation.

You are permitted to use Hugging Face libraries for infrastructure tasks such as model loading, tokenizer loading, and dataset loading, since the starter infrastructure relies on these tools. However, you may not use the Hugging Face `SFTTrainer` or similar high-level trainer APIs for the implementation component. You are required to implement the relevant training objectives yourself.

1.3.1. Supervised Fine-Tuning (SFT)

As described in Section 1.1, the first stage of most RL pipelines for language is supervised fine-tuning (SFT). SFT uses the same next-token prediction objective as pre-training, but the loss is applied only to completion tokens (i.e., no loss is applied to the prompt/query tokens). The objective is optimized over prompts x and expert completions y. In preference datasets, y is typically the preferred response y_w (sometimes referred to as Pref-FT). The objective is:

$$\max_{\theta} \mathbb{E}_{x,y\in D} \sum_{t=1}^{|y|} \log \pi_\theta(y_t|x, y_{<t}) \tag{6}$$

Action Item

Implement the supervised fine-tuning objective for a pre-trained language model. Please look at Section 4.2.2 for metrics that you would need to report for the milestone report.

SFT checkpoint for later stages. To help isolate potential bugs between your SFT implementation and your IPO/RLOO implementations, we provide an official SFT checkpoint: https://huggingface.co/asingh15/qwen-sft-countdown-defaultproj. You are welcome to use either this provided checkpoint or your own custom SFT checkpoint for your IPO and RLOO experiments.

1.3.2. Preference Optimization (IPO)

Next we will look at the implementation of the preference optimization algorithm of IPO!

Reward definition (implicit). In implicit preference optimization algorithms such as DPO and IPO, we do not train an explicit reward model; instead, the "reward" is defined implicitly from how much more likely the current policy is to produce a response than a fixed reference policy. Concretely, for a prompt x and response y, define the (scaled) per-sequence reward as

$$r_\theta(x, y) \triangleq \beta \left(\log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x)\right), \tag{7}$$

so that preference learning hopes to maximize the reward margin $r_\theta(x, y_w) - r_\theta(x, y_l)$ between the preferred and dispreferred responses.

DPO In Rafailov et al. [25], a reward reparameterization reformulates the constrained RL problem as a supervised preference-classification objective on human preference data. The DPO loss is:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]. \tag{8}$$

Here, x is the prompt, y_w is the preferred response, y_l is the dispreferred response, π_θ is the policy being optimized, and π_{ref} is the reference policy.

IPO In Gheshlaghi Azar et al. [14], the authors relax the Bradley–Terry modeling assumption to reduce overfitting to preference labels. The objective can be written as:

$$\mathcal{L}^\theta_{IPO}(\pi_\theta; \pi_{\text{ref}}) = ||h^{y_w,y_l}_{\pi_\theta} - (2\beta)^{-1}||^2_2, \quad h^{y_w,y_l}_{\pi_\theta} = \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} \tag{9}$$

Action Item

Implement the pairwise preference objective, using the SFT model as π_{ref}. This may be either the provided SFT checkpoint or your own SFT checkpoint. Please implement the IPO objective. Please look at Section 4.3.1 for metrics that you would need to report for the milestone report.

1.3.3. Online Policy-Gradient (RLOO)

Several online RL algorithms are also commonly used for fine-tuning language models. We will study one representative approach: REINFORCE Leave-One-Out (RLOO) [1]. In the default Countdown pipeline, this stage uses a rule-based verifier reward.

RLOO RLOO is a policy gradient estimator based on REINFORCE with a baseline to reduce variance of samples that is the weighted average of the reward of other samples from that policy. More formally the objective can be written as follows:

$$\frac{1}{k}\sum_{i=1}^{k}\left[R(y_{(i)}, x) - \frac{1}{k-1}\sum_{j\neq i} R(y_{(j)}, x)\right] \nabla \log \pi(y_{(i)}|x) \text{ for } y_{(1)}, ..., y_{(k)} \overset{i.i.d}{\sim} \pi_\theta(\cdot|x) \tag{10}$$

Importance Weighting In our implementation, trajectories are sampled using `vLLM` for efficiency, while the policy-gradient update is computed using the Hugging Face model that is being fine-tuned. Because these two execution paths can yield slightly different token-level log probabilities (e.g., due to kernel-specific numerics or tokenization edge cases), we treat this as sampling from a behavior policy

μ and updating a target policy π_θ. To obtain an unbiased estimator, we apply per-sample importance weighting using the ratio between the target and behavior sequence likelihoods:

$$w(y, x) = \exp\left(\log \pi_\theta(y \mid x) - \log \mu(y \mid x)\right), \tag{11}$$

and multiply the RLOO advantage term by $w(y, x)$. For stability, we compute the ratio in log-space and clip w to a maximum value.

Action Item

Implement the Online Policy-Gradient algorithm of RLOO. Please look at Section 4.3.2 for metrics that you would need to report for the milestone report.

1.4. Evaluation Setup

The next step is to use the provided evaluation pipeline to test the efficacy of preference-based and verifier-based algorithms.

Model Sampling We utilize `vLLM` (https://github.com/vllm-project/vllm) to make it more efficient for you to sample during evaluation. With an engine like this, you should find more throughput than using Hugging Face's sampling implementation of `model.generate()`.

Evaluating Countdown The default project code evaluates Countdown by sampling multiple responses per prompt and scoring each response with the local verifier in `evaluation/countdown.py`. This verifier checks both output format and arithmetic correctness.

Current Evaluation Procedure

1. Collect a set of prompts to evaluate
2. For each prompt, sample K responses from the model (default scripts use `vLLM`)
3. Score each response with the Countdown verifier
4. Report aggregate metrics such as average score and pass@K

Our goal is to compare SFT, IPO, and RLOO checkpoints under the same sampling settings.

Metric Details for Countdown Following TinyZero [21], the score has two components: (1) a format score when a parseable answer is present and (2) a verification score when the equation is correct and uses the provided numbers properly. In the starter code this corresponds to `0.0`, `0.1`, and `1.0` performance for incorrect, correctly formatted and correct responses.

Expected Performance Trends For Countdown, we expect that the warm-start (SFT) model has an average accuracy of **0.3** or higher on the test set. Both pairwise preference optimization and RLOO should improve over the SFT initialization and should have an accuracy of **0.4** and **0.5** or higher on the test set. Because `vLLM` sampling relies on a random seed and can introduce slight variation in outputs, the autograder includes a 5% margin of error. For grading, this adjusts the performance thresholds

to **0.25** for SFT, **0.35** for IPO, and **0.45** for RLOO. The score for the performance component of the milestone will be exactly:

$$\min\left(\frac{\text{accuracy}_{\text{method}}}{\text{threshold}_{\text{method}}}, 1.0\right) \times 100\%$$

Sampling Criterion For training, please use the constrained decoding sampling parameters for `vLLM` of temperature 1.0, top_k -1 (disabled), top_p 1.0 (disabled), min_p 0.0 (disabled). For evaluation, please use the constrained decoding sampling parameters for `vLLM` of temperature 0.6, top_k 20, top_p 0.95, min_p 0.0 (disabled).

2. Extension to the RL Methods Explored (Difficulty: High, Time: High)

Having explored several common policy gradient objectives used for fine-tuning language models, you are now expected to choose and investigate at least one extension. This part of the assignment is intended to be exploratory: you should implement or study a promising modification, evaluate its effect, and summarize your findings in your report. You will not be evaluated strictly on the final performance of your approach. Instead, grading will focus on your methodology, specifically the series of ideas you attempt and how thoroughly you document them. While the core implementation portion reduces the overall burden of designing the project from scratch, there is still a baseline novelty requirement for the extension itself. Below, we provide a list of possible extension directions that you may choose from.

2.1. Synthetic Data Augmentation & Pre-training Initialization

Data quality and diversity are critical bottlenecks in modern RL systems. Two primary failure modes in RL optimization include (1) poor initialization of the policy (Base/SFT model), which severely caps downstream reasoning capabilities (e.g., the 'aha' moment discussed in DeepSeek-AI [8]), and (2) a sharp reduction in dataset diversity as RL training progresses, leading to mode collapse and capacity challenges. One powerful approach to mitigating this is leveraging synthetic data augmentation to curate diverse preferences. Alternatively, you might investigate how early RL interventions or information-gain objectives during intermediate pretraining can establish a better foundation before standard SFT.

Some references to consider are Dong and Ma [9], Lee et al. [19], Yang et al. [36], Hatamizadeh et al. [16], Bansal et al. [2].

2.2. Improving Efficiency through Off-Policy Sampling

Online policy-gradient objectives, such as RLOO or GRPO, are typically trained strictly on-policy, requiring fresh environment rollouts for every gradient update. In domains where environment sampling is computationally expensive or latency-bound (e.g., agentic coding tasks or robotics), extensive on-policy sampling becomes prohibitive. Consider relaxing this constraint by exploring value-based or off-policy RL algorithms that allow for greater sample reuse, or rethinking the core RL formulation entirely from a maximum-likelihood perspective.

Some references to consider are Jia et al. [17], Tang et al. [33], Bartoldson et al. [3], Tajwar et al. [32].

2.3. Incorporating Test Time Inference

Once a policy is trained, performance can be further scaled by offloading computation to test-time inference. Explore strategies that make the policy more robust during decoding, such as utilizing generative verifiers (where reward modeling is framed as next-token prediction) or coordinating multi-model reasoning trajectories. You might also explore recursive decomposition strategies that allow models to inspect and break down extremely long contexts during inference.

Some references to consider are Zhang et al. [38], Chen et al. [6], Snell et al. [28], Zhang et al. [37].

2.4. Exploration in Discrete Token Spaces

Language-based RL frequently suffers from exploration inefficiencies due to the vast, combinatorial nature of the discrete token action space. Consider designing novel exploration strategies tailored specifically for language models. This could involve leveraging intrinsic motivation signals—such as epistemic uncertainty estimation, novelty detection in the semantic embedding space, or rethinking entropy regularization for computationally bounded agents—to encourage the policy to discover diverse reasoning paths.

Some references to consider are Qu et al. [24], Xie et al. [35], Foster et al. [11], Finzi et al. [10].

2.5. Self-Play & Multi-Agent Co-evolution

A fundamental challenge in training reasoning LLMs is the scarcity of high-quality, verifiable training data. While standard RL partially mitigates this by alternating between generation and fine-tuning on correct solutions, performance often plateaus due to sparse rewards. To continually improve models beyond their initialization, investigate multi-agent RL and self-play paradigms. By constructing cooperative or adversarial frameworks agents can autonomously generate increasingly complex scenarios and progressively challenge one another. This can even extend beyond pure text to agentic generation in multi-agent simulated environments.

Some references to consider are Chen et al. [7], Dong and Ma [9], Subramaniam et al. [30], Pfaff et al. [23].

2.6. Effective Tool-Integrated Reasoning

External tools—such as code interpreters, calculators, and search engines—fundamentally expand an LLM's problem-solving support by breaking the constraints of purely text-based reasoning. However, teaching models when and how to autonomously invoke these tools remains an open challenge. Explore how RL can be used to optimize Tool-Integrated Reasoning (TIR). You could explore how execution-grounded text feedback, or procedurally generated, containerized terminal environments provide rich, scalable supervision to dynamically alter the reasoning trajectory.

Some references to consider are Zhao et al. [39], Guo et al. [15], Jin et al. [18], Gandhi et al. [13], Si et al. [27], Song et al. [29].

2.7. Learning with a Curriculum

Curriculum learning enhances training efficiency by progressively exposing the model to increasingly complex tasks, preventing it from being overwhelmed by sparse rewards early in optimization. Consider implementing an adaptive curriculum where the complexity of prompts dynamically shifts based on the policy's current success rate. For instance, self-improvement frameworks where a

"teacher" model generates synthetic, solvable stepping-stone problems for a "student" can successfully bypass initial zero-success plateaus.

Some references to consider are Parashar et al. [22], Chen et al. [5], DeepSeek-AI [8], Sundaram et al. [31].

2.8. Choose Your Own Adventure

Consider an extension of your choosing that would ultimately lead to better downstream performance on either task: Math Reasoning (Countdown).

Action Item

Brainstorm and implement an extension direction that may improve the performance, efficiency, robustness, or analysis of SFT, IPO, and RLOO. Above, we present a set of possible extension directions to consider for the default project. A negative or mixed result can still be successful if the methodology is thoughtful and the attempted ideas are documented clearly. Feel free to stop by the office hours of any of the TAs to discuss any of the directions presented here.

3. Important Information

3.1. Project Mentors

All final projects will be assigned **a single mentor CA** to serve as a point of contact. Your project mentor should be your primary point of contact throughout the quarter as you work on your project. This individual will also be the person who ultimately grades your project milestone and final reports. Once your project mentor has been assigned, you should not visit other CAs at office hours to discuss your final project since much of your time will likely be spent simply bringing them up to speed; instead, **you should visit your project mentor's office hours for project-related issues and challenges**.

In your response to the project survey, you may choose to indicate specific members of the teaching staff who you think complement your interests and/or are well-suited to guide your particular proposed project. To help you with this, listed below are the broad specialties of each CA:

- **Abhijnya Bhat**: RL, Generative models
- **Alex Nam**: RL/LLM
- **Anikait Singh**: RL/LLM
- **Anubha Mahajan**: RL, Robot learning
- **Anushree Aggarwal**: RL/LLM
- **Ethan Hsu**: LLM
- **Ifdita Hasan Orney**: RL/LLM
- **Jonathan Yang**: IL, robot learning
- **Joonwon Kang**: Robot Learning, IL, RL, Safety
- **Joy He-Yueya**: RL/LLM
- **Ke Wang**: Robotics
- **Marcel Torne**: RL, robot learning
- **Max Du**: Robot learning, IL, RL
- **Megha Srivastava**: RL/LLM, HRI
- **Perry Dong**: RL
- **Rahul Chand**: RL/LLM, Robotics
- **Riya Karumanchi**: RL
- **Shengqu Cai**: Generative Models, Infra
- **Tian Gao**: Robot learning, IL, RL
- **Tyler Lum**: RL, robot learning

Clearly, this list of reinforcement-learning topics is **not exhaustive** and, furthermore, each CA likely has expertise that extends beyond what is listed. That said, even if you find yourself matched with a project mentor whose specialties do not overlap with your chosen project area, your mentor is still broadly well-versed in reinforcement learning, well aware of what constitutes a solid final project for the course, and is ready to help you to the best of their abilities. We anticipate that some final projects will fall outside our areas of expertise as a teaching staff, and that is okay so long as there is clear connection to and application of core course topics.

You should think of your project mentor as an individual who can provide you with feedback as you hit checkpoints in your final project; it would not be in your best interest to seek out advising from your project mentor on a week-by-week basis. Instead, **consult your mentor when you have uncertainty on the path ahead or if there are major decisions to be made** that can dramatically impact the contents of your final report.

3.2. Project Groups

Final projects for the course may be completed by **individual students or in teams of no more than 3 people**. We encourage everyone to work in groups with the expectation that the overall contributions of the final project be commensurate with the group size.

While the project survey will ask that your group for CA matching purposes, project groups are mutable up to and until the deadline of the project proposal. In other words, **your group is locked in once you submit your project proposal.**

3.3. Contributions

To help us (and yourselves) keep track of individual workloads, we ask that project proposals include **an anticipated breakdown of individual roles and responsibilities**. We will also ask for your final report to include a **similar breakdown of individual contributions** as well. It is perfectly fine for roles to adapt and change over the course of the project including deleted roles and newly added roles between the proposal and final report. If there are any changes in responsibilities, we ask that they be documented in the final report so we can ensure an equitable division of labor among group members.

Project groups are welcome to work with people not enrolled in the class, however we expect the contribution breakdowns in the project proposal and final report to reflect the work of all members involved (both internal and external). Final projects may be shared between CS224R and another class however, in this case, you should clearly indicate this in your project proposal and we will expect a more ambitious project.

3.4. Honor Code and AI Tools Policy

You should work as a team independently and ensure academic integrity by clearly documenting which parts of your code or ideas were assisted by AI tools and which parts were developed independently.

For the **default project**: You are **not allowed to collaborate with AI tools** such as GitHub Co-Pilot and ChatGPT on ANY component of the default project. The only exception to this is for the extension.

Note: TAs may not look at students' code during office hours for either the default or custom final projects.

3.5. Submission Format

We recommend submitting proposals and reports using the provided LaTeX templates below. This ensures consistent formatting for all submissions.

- Project Proposal: CS224R Project Proposal Template
- Milestone: CS224R Project Milestone Template
- Final Project: CS224R Final Report Template

For poster presentations, you are free to choose templates that effectively communicate your research findings, such as Stanford-LaTeX-Poster-Template. All submissions must adhere to the specified page limits in the instructions.

Clarifications This handout is intended to be the central source for default project requirements, including submission checklists and milestone expectations. If course staff posts a clarification on Ed, we will aim to incorporate it here so students do not need to search across separate threads to understand what to submit.

3.6. Using Late Days

With the exception of the project poster (which **cannot be submitted late** due to the live poster session) and the final project report (which **cannot be submitted late** due to the University deadline to submit final grades for the quarter), entire project groups may apply a maximum of two late days to any other graded component of the final project, for example, project milestone. Groups cannot aggregate collective late days and late days will be applied to all group members; this means that, to apply two late days, each group member must have two unused late days to spare. Assignment extensions granted by the Office of OAE do not apply to group assignments (specifically the poster and final report).

3.7. Computing Resources

We will be providing **$500 in cloud computing credits** from Modal to each student for use on homework assignments and the final project, which should amount to more than 100 hours of GPU time (and much more CPU time), depending on the size of the GPU. Details about how to access these credits are forthcoming.

Avoiding unnecessary Modal rebuilds If Modal rebuilds your image on every run, it is likely capturing your local virtual environment (`venv`) in the build context. Because `venv` files change frequently, this constantly invalidates the build cache. To fix this, move your `venv` outside the project directory and source it from the new location.

4. Deliverables and Grading

The default project has the same deliverables as the custom project. The project accounts for **35**% of the course grade, with the following breakdown:

- Survey – 0%
- Project proposal (+SFT Implementation) – 4%

- Milestone (IPO + RLOO) – 5%
- Poster presentation – 8%
- Final project report – 18%

4.1. Survey:

In the survey, indicate your project group and confirm that your group is completing the default project (not a custom project).

4.2. Project Proposal + SFT Implementation (5/1):

The first checkpoint is a joint submission of two components: (1) the project proposal and (2) the milestone of your SFT implementation. This checkpoint is split across the following Gradescope submissions:

1. **Default Project Proposal + SFT Implementation:** Submit a single PDF containing the project proposal and the SFT implementation results described below.
2. **Default Project SFT Milestone:** Submit two files: `sft.py`, which contains your SFT implementation, and `eval.json`, which contains the output of the evaluation script used to compute pass@k.

4.2.1. Project Proposal

Your proposal should focus on your chosen extension. Describe a concrete idea or technique that could help address the extension, and provide evidence or motivation for why this approach is promising.

Your proposal should have the following structure:

1. **Extension** (1/4 page). Describe the extension you plan to pursue for the default project and why it is relevant and important.
2. **Related Work** (1/2 page). Summarize the most relevant prior work and explain how it falls short of achieving the extension described above.
3. **Technical Outline of Extension** (1/2 page). Provide a high-level description of your approach, clearly identifying the novel technical contribution and your evaluation plan.

The proposal is where you will receive feedback on your extension idea, related work, and technical plan.

4.2.2. SFT Implementation:

The SFT component of the report must include the evaluation metrics below to demonstrate the effectiveness of your implementation.

Metrics for SFT Report the following metrics. All reported metrics should be included in the PDF as plots, for example generated with matplotlib, rather than only as scalar values.

- **Cross Entropy loss on Train/Test set over training iterations:** Measures the negative log-likelihood of the target tokens. This indicates how well the model is learning the target distribution and predicting the exact reference text.

- **Token Accuracy on Train/Test set over training iterations:** The percentage of times the model's highest-probability next token exactly matches the ground-truth token in the dataset.
- **Pass at k for final evaluation checkpoint on Test Set:** Evaluates functional correctness. It measures the probability that at least one out of k generated candidate responses successfully passes the required correctness checks or unit tests.

Additionally, please include one completion from the LLM for a sample test problem in the PDF. The separate Default Project SFT Milestone submission must contain `sft.py` and `eval.json`; no additional report is required for that separate submission.

Submission & Grading – Submit one proposal PDF per group to Gradescope under the "Project Proposal + SFT Implementation" assignment. The PDF must include the finalized list of group members, the project proposal with an extension, related works, and technical outline, the SFT metrics listed above, and one sample completion. Submit `sft.py` and `eval.json` separately under the "Default Project SFT Milestone" assignment. Although the proposal deadline is 5/1, we strongly encourage you to **start developing your project idea early**. The proposal will be graded primarily to provide feedback and help you adjust (or pivot) your plan to meet the project requirements.

4.3. IPO + RLOO Milestone (5/22)

Submit one milestone report PDF per group (including the names of all project members) to Gradescope under the assignment "IPO/RLOO Milestone Submission PDF." There are two components for the report that must be submitted: (1) IPO and (2) RLOO. More details are below.

Students do not need to submit anything additional for the extension in the default project milestones. The extension component is covered in the Default Project Proposal, where you will receive feedback on your extension idea, related work, and technical plan. The IPO/RLOO milestone template can be used as is.

Submission Checklist – The IPO/RLOO milestone is split across the following Gradescope submissions:

1. **IPO Milestone assignment:** Submit two files: `ipo_trainer/ipo.py`, which contains your IPO implementation, and `eval.json`, which contains the output of the evaluation script used to compute pass@k.
2. **RLOO Milestone assignment:** Submit two files: `rloo_trainer/rloo_update_worker.py`, which contains your RLOO implementation, and `eval.json`, which contains the output of the evaluation script used to compute pass@k.
3. **IPO/RLOO Milestone Submission PDF:** Submit one PDF containing the IPO and RLOO reporting items below.

The IPO/RLOO Milestone Submission PDF must include:

- **IPO:** IPO loss calculation, IPO reward margin calculation, IPO pass@k, and one IPO rollout.
- **RLOO:** RLOO loss calculation, importance weight during training, KL loss during training, rollout accuracy on the test set over training iterations, RLOO pass@k, and one RLOO rollout.
- **Contributions Statement**: Updated breakdown of each member's contributions so far and planned contributions for the remaining work.

For the PDF, all reported metrics should be shown as plots, for example generated with matplotlib, rather than only as scalar values.

4.3.1. IPO Milestone Component

The IPO component of the report must include the evaluation metrics below to demonstrate the effectiveness of your implementation.

Metrics for IPO Report the following metrics (use matplotlib to generate conference-style plots):

- **IPO loss on Train/Test set over training iterations:** This demonstrates how well the model is minimizing the regularized preference loss between chosen and rejected completions without needing a separate reward model.
- **IPO Reward Margin on Train/Test set over training iterations:** The difference in implicit reward (or log-probability ratios) between the chosen and rejected responses. An increasing margin indicates the model is successfully learning to separate and prefer high-quality responses.
- **Pass at k for final evaluation checkpoint on Test Set:** Evaluates the functional correctness post-alignment, ensuring the preference optimization hasn't degraded the model's actual problem-solving capabilities.

Additionally, include one test-set rollout for a sample test problem in the PDF.

4.3.2. RLOO Milestone Component

The RLOO component of the report must include the evaluation metrics below to demonstrate the effectiveness of your implementation.

Metrics for RLOO Report the following metrics (use matplotlib to generate conference-style plots):

- **RLOO loss on Train set over training iterations:** The policy gradient loss utilizing the REINFORCE Leave-One-Out baseline. This tracks the optimization of the expected reward while controlling for gradient variance.
- **Importance Weight Mean on Train set over training iterations:** Tracks the ratio of probabilities between the current policy and the reference policy. Monitoring this ensures the active policy is updating smoothly and not diverging too aggressively, vital for training stability.
- **KL Loss on Train set over training iterations:** The Kullback-Leibler divergence penalty between the active policy and the initial SFT model. This metric is crucial for verifying that the model is not "reward hacking" or suffering from catastrophic forgetting of its foundational language modeling priors.
- **Rollout Accuracy (over 16 samples) on the Train Set over training iterations:** The average correctness of the 16 generated samples per prompt during the RL phase. This provides a stable, low-variance estimate of how the policy's actual task performance is evolving.
- **Pass at k for final evaluation checkpoint on Test Set:** The ultimate functional correctness metric, confirming the problem-solving ability of the fully RLOO-aligned model.

Additionally, include one test-set rollout for a sample test problem in the PDF.

4.4. Poster Presentation (6/3)

Prepare a poster that **focuses on the extension explored in Section 2**. Your poster should address the following:

1. What is the problem?
2. Why is the problem interesting and important?
3. What are the key findings from my experiments?
4. What can audiences learn from this work? What is the limitation / future work?

Poster Dimensions – We recommend a landscape poster of size 24" $\times$ 36". Other sizes are acceptable, but this size fits best on the poster boards and easels provided at the session; please avoid posters that are substantially larger or smaller.

Poster Printing – Commercial poster printing is optional. You may also print the poster on smaller sheets (letter or A4) and tape them together; Adobe Acrobat Reader supports this via the "Poster" option in the Print dialog. You are responsible for printing your poster, so please plan ahead. Below are some options for 20" $\times$ 30" printing; verify turnaround times and services yourself (the information below is compiled from the providers' websites).

Some resources for printing:

- Lathrop Library Tech Desk: order online for a 3-day turnaround.
- FedEx: approximately 2-day turnaround. Offers customizability (e.g. laminate the poster for greater durability; add grommets).
- CVS Photo: Affordable, same-day pickup on some options; can use code 50SITE for offer.
- Walgreens: claims to have same-day pickup for 24" $\times$ 36".
- Biotech Productions: specialized for research posters and offers same-day free delivery to Stanford campus.
- Walmart: Different printing options and templates; home-delivery option

Poster Exemptions – At least one group member must attend the poster session, unless you have received explicit permission from the course staff (via email) to submit a video presentation instead. The only automatic exception is for groups composed entirely of (remote) SCPD students; these groups are expected, by default, to submit a five-minute poster video. Upload a Google Drive or YouTube link to Gradescope under the "Project Poster Video" assignment.

Location & Time – The poster session will be held on 6/4/25, 9a.m-3p.m at the Burnham Pavilion.

4.5. Report (6/8)

The final report should be in the style of a research paper. It should be preceded by a one-page extended abstract and should include the sections below. The final report will be graded using a 20-point rubric:

1. **One-page extended abstract (2 pts):** Include this as the first page of the full report. It should function as a standalone summary and clearly cover: (1) motivation and problem statement, (2) method and novelty, (3) implementation details and headline results, and (4) discussion, limitations, and conclusion.
2. **Abstract (0.5 pts):** Summarize the problem, method, key result, and significance in a single coherent paragraph.
3. **Introduction (2 pts):** Provide problem background, motivate the project using real applications and/or literature, and state explicit research questions or hypotheses.
4. **Related Work (2 pts):** Cover the relevant prior work, position your project clearly relative to it, and ground your novelty claim in cited literature.

5. **Method (5 pts):** Describe the extension in enough technical detail that a reader could reproduce it from the text alone. Justify all major components, state assumptions, include clear pseudo-code or a method diagram when helpful, and explain what is novel relative to prior work.
6. **Experimental Setup (1 pt):** Clearly describe the dataset/task, baselines, evaluation metrics, key hyperparameters, and training details. Justify baseline and metric choices.
7. **Results (6 pts):** Include both quantitative and qualitative analysis.
    - **Quantitative evaluation (3 pts):** Report complete tables/figures for the promised comparisons, include variance/error bars or multiple seeds when feasible, and explain what the numbers show.
    - **Qualitative analysis (3 pts):** Include examples such as rollouts, ablations, failure cases, qualitative trajectories, or visualizations, and discuss why the method works or fails.
8. **Discussion (0.5 pts):** Discuss limitations, broader impacts, and difficulties encountered.
9. **Conclusion (0.5 pts):** Provide an insightful take-home message, explain the significance of your findings, and identify specific future directions.
10. **Team Contributions (0.5 pts):** Provide an equitable breakdown of individual contributions and explicitly document and justify any changes from the proposal.

Formatting & Contributions – Strong reports typically have a main body of about eight pages, but there are no hard length or formatting requirements beyond the one-page extended abstract. Project groups should include an updated breakdown of individual contributions and briefly explain any changes relative to the plan described in the project proposal.

Submission & Grading – Submit one final report per group with the names of all project members to Gradescope under the assignment “Final Project Report.”

5. Expectation of the Default Project

The default project requires implementing, evaluating, and documenting **a novel extension** for the RL Fine-Tuning of Language Models. Note that the extension is 50% of the grade in this assignment and requires careful thought into its construction.

The *novelty* should entail exploration of an **unanswered or partially answered** problem (potentially guided by the sample extension directions presented in Section 2). This may include exploring or developing a new algorithmic idea that can lead to better performance, higher efficiency, or instill desirable properties into the language model that were not present before. Often students think that the goal is to build the highest performing model instead of doing "science" to figure out strengths and weaknesses of whatever they have tried. Therefore, achieving state-of-the-art performance is not required for this project and our performance requirements will be very lax for the extension. We will place more weight on the quality of your methodology, the sequence of ideas you tried, and how clearly you document the outcomes than on the final score alone.

In light of the above goals, students are expected to prepare a proposal, prepare a milestone report, present their work in the final poster session, and submit the final report, with specific details as outlined in this document.

6. Summary

Congratulations on making it this far! We hope you gained some understanding of how RL algorithms should be implemented for a large language model. Additionally, we hope that the extension provides some insight on some of the different design decisions that go into algorithm design and/or

implementation. Finally, the implementation of the evaluation framework would allow you to gain some insight on building similar data pipelines in the future.

References

[1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 2024. URL https://arxiv.org/abs/2402.14740.

[2] Rachit Bansal, Clara Mohri, Tian (Sunny) Qin, David Alvarez-Melis, and Sham Kakade. Rl excursions during pretraining: How early is too early for on-policy learning?, 2026. URL https://rl-excursions.github.io/.

[3] Brian R. Bartoldson, Siddarth Venkatraman, James Diffenderfer, Moksh Jain, Tal Ben-Nun, Seanie Lee, Minsu Kim, Johan Obando-Ceron, Yoshua Bengio, and Bhavya Kailkhura. Trajectory balance with asynchrony: Decoupling exploration and learning for fast, scalable llm post-training, 2025.

[4] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. ISSN 00063444. URL http://www.jstor.org/stable/2334029.

[5] Xiaoyin Chen, Jiarui Lu, Minsu Kim, Dinghuai Zhang, Jian Tang, Alexandre Piché, Nicolas Gontier, Yoshua Bengio, and Ehsan Kamalloo. Self-evolving curriculum for llm reasoning, 2025. URL https://arxiv.org/abs/2505.14970.

[6] Xinyun Chen et al. Can llms collaborate on reasoning trajectories? *arXiv preprint arXiv:2510.06410*, 2025.

[7] Yixing Chen et al. Multi-agent evolve: Llm self-improve through co-evolution. *arXiv preprint arXiv:2510.23595*, 2025.

[8] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[9] Kefan Dong and Tengyu Ma. Stp: Self-play llm theorem provers with iterative conjecturing and proving, 2025. URL https://arxiv.org/abs/2502.00212.

[10] Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J. Zico Kolter, and Andrew Gordon Wilson. From entropy to epiplexity: Rethinking information for computationally bounded intelligence, 2026. URL https://arxiv.org/abs/2601.03220.

[11] Dylan J. Foster, Zakaria Mhammedi, and Dhruv Rohatgi. Is a good foundation necessary for efficient reinforcement learning? the computational role of the base model in exploration, 2025. URL https://arxiv.org/abs/2503.07453.

[12] Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. Stream of search (sos): Learning to search in language, 2024. URL https://arxiv.org/abs/2404.03683.

[13] Kanishk Gandhi, Shivam Garg, Noah D. Goodman, and Dimitris Papailiopoulos. Endless terminals: Scaling rl environments for terminal agents, 2026. URL https://arxiv.org/abs/2601.16443.

[14] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A General Theoretical Paradigm to Understand Learning from Human Preferences. *arXiv e-prints*, art. arXiv:2310.12036, October 2023. doi: 10.48550/arXiv.2310.12036.

[15] Xinyu Guo et al. Tool-star: Empowering llm-brained multi-tool reasoner via reinforcement learning. *arXiv preprint arXiv:2505.16410*, 2025.

[16] Ali Hatamizadeh, Syeda Nahida Akter, Shrimai Prabhumoye, Jan Kautz, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, and Yejin Choi. Rlp: Reinforcement as a pretraining objective, 2025. URL https://arxiv.org/pdf/2510.01265.

[17] Hao Jia et al. Trajectory bellman residual minimization: A simple value-based method for llm reasoning. *arXiv preprint arXiv:2505.15311*, 2025.

[18] Bowen Jin, Hansi Zeng, Zhenrui Yue, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning, 2025. URL https://arxiv.org/abs/2503.09516.

[19] Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback, 2024. URL https://arxiv.org/abs/2309.00267.

[20] Jincheng Mei, Wesley Chung, Valentin Thomas, Bo Dai, Csaba Szepesvari, and Dale Schuurmans. The role of baselines in policy gradient optimization. *Advances in Neural Information Processing Systems*, 35:17818–17830, 2022.

[21] Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. Tinyzero. https://github.com/Jiayi-Pan/TinyZero, 2025. Accessed: 2025-01-24.

[22] Shubham Parashar et al. Curriculum reinforcement learning from easy to hard tasks improves llm reasoning. *arXiv preprint arXiv:2506.06632*, 2025.

[23] Nicholas Pfaff, Thomas Cohn, Sergey Zakharov, Rick Cory, and Russ Tedrake. Scenesmith: Agentic generation of simulation-ready indoor scenes, 2026. URL https://arxiv.org/abs/2602.09153.

[24] Yuxiao Qu, Matthew Y. R. Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. Optimizing test-time compute via meta reinforcement fine-tuning, 2025. URL https://arxiv.org/abs/2503.07572.

[25] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

[26] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[27] Chenglei Si, Zitong Yang, Yejin Choi, Emmanuel J. Candès, Diyi Yang, and Tatsunori Hashimoto. Towards execution-grounded automated ai research, 2026. URL https://arxiv.org/abs/2601.14525.

[28] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL https://arxiv.org/abs/2408.03314.

[29] Yuda Song, Lili Chen, Fahim Tajwar, Remi Munos, Deepak Pathak, J. Andrew Bagnell, Aarti Singh, and Andrea Zanette. Expanding the capabilities of reinforcement learning via text feedback, 2026. URL https://arxiv.org/abs/2602.02482.

[30] Vighnesh Subramaniam, Yilun Du, Joshua B. Tenenbaum, Antonio Torralba, Shuang Li, and Igor Mordatch. Multiagent finetuning: Self improvement with diverse reasoning chains, 2025. URL https://arxiv.org/abs/2501.05707.

[31] Shobhita Sundaram, John Quan, Ariel Kwiatkowski, Kartik Ahuja, Yann Ollivier, and Julia Kempe. Teaching models to teach themselves: Reasoning at the edge of learnability, 2026. URL https://arxiv.org/abs/2601.18778.

[32] Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng, and Andrea Zanette. Maximum likelihood reinforcement learning, 2026. URL https://arxiv.org/abs/2602.02710.

[33] Yunhao Tang, Taco Cohen, David W. Zhang, Michal Valko, and Rémi Munos. Rl-finetuning llms from on- and off-policy data with a single algorithm, 2025. URL https://arxiv.org/abs/2503.19612.

[34] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, May 1992.

[35] Tengyang Xie, Dylan J. Foster, Akshay Krishnamurthy, Corby Rosset, Ahmed Awadallah, and Alexander Rakhlin. Exploratory preference optimization: Harnessing implicit q*-approximation for sample-efficient rlhf, 2024. URL https://arxiv.org/abs/2405.21046.

[36] Zitong Yang, Neil Band, Shuangping Li, Emmanuel Candès, and Tatsunori Hashimoto. Synthetic continued pretraining, 2024. URL https://arxiv.org/abs/2409.07431.

[37] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2025. URL https://arxiv.org/abs/2512.24601.

[38] Lunjun Zhang, Hritik Bansal, Mehran Kazemi, and Rishabh Agarwal. Generative verifiers: Reward modeling as next-token prediction. *arXiv preprint arXiv:2408.15240*, 2024.

[39] Yuhang Zhao et al. Understanding tool-integrated reasoning. *arXiv preprint arXiv:2508.19201*, 2025.